

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-acd624a7839ce563f.jsonl`  
- SHA-256: `2b46a9fc5c3ca7f823452eeb49f4de6de5f96d5dc9186ee7396af2007561a827`  
- Source modified: `2026-06-22T09:50:54+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_416cc0e5-a35`  
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`
```

Transcript

1. user (2026-06-22T09:48:54.924Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{  
  "title": "billing.WALL_SECONDS constant exists and cost_report() correctly uses it",  
  "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/billing.py:19-20, nightly_reelcount.py:73-90",  
  "severity": "low",  
  "detail": "Line 80 in nightly_reelcount.py uses billing.WALL_SECONDS key. billing.py line 19 exports WALL_SECONDS = 'wall_seconds'. The cost_report() function assembles usage and rates dicts with this key, then calls billing.compute_cost_usd(usage, rates) which handles it correctly (line 39-45 in billing.py).",  
  "fix": "No fix needed. Billing integration is correct."  
}
```

2. assistant (2026-06-22T09:48:56.982Z)

I'll review this finding by examining the relevant files to verify whether the billing.WALL_SECONDS constant is correctly used in the nightly regression engine.

3. user (2026-06-22T09:48:58.169Z)

```
1      """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM  
nightly).  
2  
3      Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a  
deployment serves  
4      it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does  
not change**  
5      vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score  
of CACHED  
6      trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real  
model*, so it
```

```

7      needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8      owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10     What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11     ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12     Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13     **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25     python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28     python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")

```

```

56     DEFAULT_QUALITY_TOL = 0.02      # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                     fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
87             "usd": usd,
88             "inr": billing.to_inr(usd, fx_inr_per_usd),
89             "breakdown": breakdown,
90         }
91
92
93     # ----- #
94     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95     # ----- #
96     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                           cost: Dict[str, Any]) -> Dict[str, Any]:
98         """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
108                 "quality": r.get("quality"),
109             }
110         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
111                 "n": len(per_video)}
112
113

```

```

114 @dataclass
115 class Finding:
116     video: str
117     metric: str          # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
118     kind: str            # "regression" | "improvement" | "stale"
119     baseline: Optional[float] = None
120     current: Optional[float] = None
121     detail: str = ""
122
123     def message(self) -> str:
124         if self.kind == "stale":
125             return f"[STALE] {self.metric}: {self.detail}"
126         if self.metric == "error":
127             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128         b = "?" if self.baseline is None else f"{self.baseline:g}"
129         c = "?" if self.current is None else f"{self.current:g}"
130         tag = "REGRESSION" if self.kind == "regression" else "improvement"
131         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134 @dataclass
135 class NightlyResult:
136     findings: List[Finding] = field(default_factory=list)
137     added: List[str] = field(default_factory=list)
138     removed: List[str] = field(default_factory=list)
139
140     @property
141     def regressions(self) -> List[Finding]:
142         return [f for f in self.findings if f.kind == "regression"]
143
144     @property
145     def improvements(self) -> List[Finding]:
146         return [f for f in self.findings if f.kind == "improvement"]
147
148     @property
149     def stale(self) -> List[Finding]:
150         return [f for f in self.findings if f.kind == "stale"]
151
152     def overall_status(self) -> str:
153         if self.regressions:
154             return "REGRESSION"
155         if self.stale:
156             return "STALE"
157         return "PASS"
158
159     def exit_code(self) -> int:
160         if self.regressions:
161             return 1
162         if self.stale:
163             return 4
164         return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """

```

```

175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                     detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}}"))
179         return res
180
181         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182         res.added = sorted(set(c_pv) - set(b_pv))
183         res.removed = sorted(set(b_pv) - set(c_pv))
184         if res.added or res.removed:
185             res.findings.append(Finding("", "corpus", "stale",
186                                     detail=f"added={res.added} removed={res.removed}"))
187
188         for v in sorted(set(b_pv) & set(c_pv)):
189             b, c = b_pv[v], c_pv[v]
190             # 1) pipeline must still succeed
191             if not c.get("ok", False) or c.get("reel_count") is None:
192                 res.findings.append(Finding(v, "error", "regression",
193                                     detail=str(c.get("error") or "no reel_count
produced"))))
194                 continue
195             # 2) reel count ? EXACT
196             bc, cc = b.get("reel_count"), c.get("reel_count")
197             if bc is not None and cc is not None and cc != bc:
198                 res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199             # 3) quality metrics ? tolerance (only if both sides have them)
200             bq, cq = b.get("quality") or {}, c.get("quality") or {}
201             for m in QUALITY_HIGHER:
202                 bv, cv = bq.get(m), cq.get(m)
203                 if bv is None or cv is None:
204                     continue
205                 if cv < bv - tols.quality_tol:
206                     res.findings.append(Finding(v, m, "regression", bv, cv))
207                 elif cv > bv + tols.quality_tol:
208                     res.findings.append(Finding(v, m, "improvement", bv, cv))
209         return res
210
211
212         # ----- #
213         # Report formatting (pure).
214         # ----- #
215         def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
216             if not qd or qd.get(key) is None:
217                 return "?"
218             return f"{qd[key]:.3f}"
219
220
221         def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                         result: NightlyResult, date: str, head: str) -> str:
223             status = result.overall_status()
224             badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225                     "STALE": "? STALE (re-freeze the baseline)"}[status]
226             cost = current.get("cost", {})
227             L: List[str] = [
228                 f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
229                 "",
230                 f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
`{current.get('segmenter')}`",
231                 "",
232                 f"- HEAD `{head}` · baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
233                 f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})** · "
234                 f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',

```

```

0)}}/hr",
235         "",
236     ]
237     if result.regressions:
238         L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
239     + [""]
240     if result.stale:
241         L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
242         L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
243     # Per-video table.
244     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245     L += ["## Per-video", ""]
246         "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247         "|---|---|---|---|---|"
248     for v in sorted(set(b_pv) | set(c_pv)):
249         b, c = b_pv.get(v, {}), c_pv.get(v, {})
250         bc = b.get("reel_count")
251         cc = c.get("reel_count")
252         rc = f"'?' if bc is None else bc?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253         bq, cq = b.get("quality"), c.get("quality")
254         tf = f"_q(bq, 'tol_f1')?_q(cq, 'tol_f1')?"
255         f5 = f"_q(bq, 'f1@0.5')?_q(cq, 'f1@0.5')?"
256         vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
257         L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258     L.append("")
259
260     if result.improvements:
261         L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
+ [f"- {f.message()}" for f in result.improvements] + [""]
262     L += ["---",
263         "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
264         "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
265     return "\n".join(L)
266
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:
289                     out.append((float(row[0]), float(row[1])))
290                 except (ValueError, IndexError):

```

```

291         continue
292     return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(start, end) pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (`start_time`/`end_time` ? motion/DB shape ? or `start`/`end`).
Used only for the
298         optional quality metrics; the reel COUNT is `len(segs)` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                     rerun_on_mismatch: bool = True,
311                     baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
`add_play_segment`?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)

```

```

346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")
349         segs = result.value if hasattr(result, "value") else result
350         intervals = _segments_to_intervals(segs)
351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
value
367
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")}
372                 if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402
403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
405         """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""

```



```

406     parts = dotted.split(".")
407     obj = config
408     for p in parts[:-1]:
409         obj = getattr(obj, p, None)
410         if obj is None:
411             return
412     try:
413         setattr(obj, parts[-1], value)
414     except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
415         pass
416
417
418     def _git_head() -> str:
419         import subprocess
420         try:
421             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
422                                 capture_output=True, text=True, timeout=10)
423             return out.stdout.strip() or "?"
424         except Exception: # noqa: BLE001
425             return "?"
426
427
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):
430             return None
431         with open(path, encoding="utf-8") as fh:
432             return json.load(fh)
433
434
435     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
436         out = dict(snapshot)
437         out["generated_at"] = _dt.date.today().isoformat()
438         out["git_commit"] = _git_head()
439         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                       "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                       "configured+checkpointed engine + the manifest corpus.")
442         out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443         os.makedirs(os.path.dirname(path), exist_ok=True)
444         tmp = path + ".tmp"
445         with open(tmp, "w", encoding="utf-8") as fh:
446             json.dump(out, fh, indent=2, sort_keys=True)
447             fh.write("\n")
448         os.replace(tmp, path)
449
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454         p.add_argument("--baseline", default=DEFAULT_BASELINE)
455         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457         p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                       help="authoritative VM uptime (boot?delete) for the cost line; "
459                       "defaults to the in-process pipeline wall time (a lower bound)")
460         p.add_argument("--no-rerun-guard", action="store_true",
461                       help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462         p.add_argument("--update-baseline", action="store_true",
463                       help="freeze the current reel counts + quality as the new baseline
instead of checking")
464         p.add_argument("--email", action="store_true", help="email the report (see

```

```

scripts/nightly_email.py"))
465     p.add_argument("--email-to", default=None, help="override the report recipient")
466     p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467     p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468     return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487         except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488             log.error("run failed: %s", e)
489             return 2
490         if not run_results:
491             log.error("no videos in manifest %s", args.manifest)
492             return 2
493
494         billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495         cost = cost_report(
496             billed_seconds=billed,
497             machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498             fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499             gpu_seconds=total_wall,
500         )
501         segmenter = manifest.get("segmenter", "motion")
502         snapshot = summarize_results(run_results, segmenter, cost)
503         snapshot["git_commit"] = _git_head()
504         snapshot["generated_at"] = date
505
506         if args.update_baseline:
507             save_baseline(args.baseline, snapshot)
508             log.info("baseline frozen ? %s", args.baseline)
509             for r in run_results:
510                 log.info(" %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511             return 0
512
513         if baseline is None:
514             log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515             return 3
516
517         result = compare(baseline, snapshot, tols)
518         report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520         if not args.no_report:

```

```

521         os.makedirs(args.report_dir, exist_ok=True)
522         md_path = os.path.join(args.report_dir, f"{date}.md")
523         with open(md_path, "w", encoding="utf-8") as fh:
524             fh.write(report + "\n")
525         with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526             json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                        "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                        fh, indent=2, sort_keys=True)
529         log.info("report ? %s", md_path)
530
531     print(report)
532
533     if args.email:
534         try:
535             from scripts.nightly_email import send_report
536             subject = f"[KheISutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537             sent = send_report(subject, report, to=args.email_to)
538             log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539         except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540             log.error("email failed (report still written): %s", e)
541
542     code = result.exit_code()
543     if code == 1:
544         log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545     elif code == 4:
546         log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547     else:
548         log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549     return code
550
551
552 if __name__ == "__main__":
553     raise SystemExit(main())
554

```

4. user (2026-06-22T09:48:58.310Z)

```

1     """Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
2
3     ``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
4     (the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
**USD**
5     (matches GCP billing = one source of truth) and **displayed in INR** at a configurable
FX rate
6     (the Bangalore market). The pricebook is a versioned DB table
(``backend/infrastructure/database.py``);
7     this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates
dict, so it
8     is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
9
10    **Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every
cost is
11    ``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated
pricebook version.
12    """
13

```

```

14     from __future__ import annotations
15
16     from typing import Dict, Mapping, Tuple
17
18     # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
breakdown.
19     GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second,
per gpu_type
20     WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
billed as GPU)
21     EGRESS_GB = "egress_gb"          # reel/byte egress in GB
22     GEMINI_USD = "gemini_usd"        # provider-reported cost, ALREADY in USD (a passthrough,
not rate-x)
23
24     # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
USD amount).
25     _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
26
27
28     def compute_cost_usd(usage: Mapping[str, float],
29                           rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
30         """Return ``(cost_usd, breakdown)`` for one job.
31
32         ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
a resource
33         key to its USD unit price (already resolved for this pricebook version + gpu_type by
the DB
34         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
35         Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
is ``0.0``.
36         """
37         breakdown: Dict[str, float] = {}
38         total = 0.0
39         for res in _RATE_MULTIPLIED:
40             amount = float(usage.get(res, 0.0) or 0.0)
41             rate = float(rates.get(res, 0.0) or 0.0)
42             cost = round(amount * rate, 6)      # round the LINE, then sum rounded lines so
the
43             if amount or rate:                  # breakdown reconciles to the total exactly
(no drift),
44                 breakdown[res] = cost          # and record the line even at $0 for
transparency
45             total += cost
46         gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
47         if gemini:
48             breakdown[GEMINI_USD] = gemini
49         total += gemini
50         return round(total, 6), breakdown
51
52
53     def apply_margin(cost_usd: float, margin: float) -> float:
54         """Resale price = ``cost_usd × (1 + margin)``. ``margin`` is clamped ? 0 (never
below cost)."""
55         return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
58     def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
59         """Display conversion: USD ? INR at the configured FX rate (2 dp, the display
currency)."""
60         return round(float(cost_usd) * float(fx_inr_per_usd), 2)
61

```

Now let me verify the specific claim about the billing constant and its usage:

6. user (2026-06-22T09:49:02.284Z)

```
20:WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
billed as GPU)
25:_RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
```

7. user (2026-06-22T09:49:03.279Z)

```
80:     usage = {billing.WALL_SECONDS: float(billed_seconds)}
81:     rates = {billing.WALL_SECONDS: rate_per_sec}
82:     usd, breakdown = billing.compute_cost_usd(usage, rates)
```

8. assistant (2026-06-22T09:49:05.335Z)

Perfect. Now let me verify the compute_cost_usd function handles WALL_SECONDS correctly:

9. user (2026-06-22T09:49:06.313Z)

```
28     def compute_cost_usd(usage: Mapping[str, float],
29                           rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
30         """Return ``(cost_usd, breakdown)`` for one job.
31
32         ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
a resource
33         key to its USD unit price (already resolved for this pricebook version + gpu_type by
the DB
34         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
35         Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
is ``0.0``.
36         """
37         breakdown: Dict[str, float] = {}
38         total = 0.0
39         for res in _RATE_MULTIPLIED:
40             amount = float(usage.get(res, 0.0) or 0.0)
41             rate = float(rates.get(res, 0.0) or 0.0)
42             cost = round(amount * rate, 6)    # round the LINE, then sum rounded lines so
the
43             if amount or rate:                # breakdown reconciles to the total exactly
(no drift),
44                 breakdown[res] = cost        # and record the line even at $0 for
transparency
45             total += cost
46         gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
47         if gemini:
48             breakdown[GEMINI_USD] = gemini
49         total += gemini
50         return round(total, 6), breakdown
51
52
53     def apply_margin(cost_usd: float, margin: float) -> float:
54         """Resale price = ``cost_usd * (1 + margin)``. ``margin`` is clamped ? 0 (never
below cost)."""
55         return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
```

10. assistant (2026-06-22T09:49:08.665Z)

Excellent. Now let me check the context around line 25 in billing.py to confirm _RATE_MULTIPLIED includes WALL_SECONDS:

11. user (2026-06-22T09:49:09.559Z)

```
14     from __future__ import annotations
15
16     from typing import Dict, Mapping, Tuple
17
18     # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
breakdown.
19     GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second,
per gpu_type
20     WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
billed as GPU)
21     EGRESS_GB = "egress_gb"          # reel/byte egress in GB
22     GEMINI_USD = "gemini_usd"        # provider-reported cost, ALREADY in USD (a passthrough,
not rate-x)
23
24     # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
USD amount).
25     _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
26
27
28     def compute_cost_usd(usage: Mapping[str, float],
29                           rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
30         """Return ``(cost_usd, breakdown)`` for one job.
31
32         ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
a resource
33         key to its USD unit price (already resolved for this pricebook version + gpu_type by
the DB
34         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
35         Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
is ``0.0``.
36         """
37         breakdown: Dict[str, float] = {}
38         total = 0.0
39         for res in _RATE_MULTIPLIED:
```

12. assistant (2026-06-22T09:49:12.071Z)

Perfect. Now let me verify the integration more carefully by checking the usage flow in nightly_reelcount.py around the cost_report function:

13. user (2026-06-22T09:49:12.947Z)

```
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                     fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
```

```
87         "usd": usd,
88         "inr": billing.to_inr(usd, fx_inr_per_usd),
89         "breakdown": breakdown,
```

14. assistant (2026-06-22T09:49:15.035Z)

Now let me verify the import of billing at the top and check that to_inr exists:

15. user (2026-06-22T09:49:16.627Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/billing.py:58:def to_inr(cost_usd: float,
fx_inr_per_usd: float) -> float:
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/nightly_reelcount.py:88:         "inr":
billing.to_inr(usd, fx_inr_per_usd),
```

16. assistant (2026-06-22T09:49:19.396Z)

Perfect. Now let me verify the import statement and the overall correctness:

17. user (2026-06-22T09:49:20.580Z)

```
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
```

18. assistant (2026-06-22T09:49:23.281Z)

Excellent. Now let me do a final verification to confirm the constant value and its integration into the compute_cost_usd flow:

19. user (2026-06-22T09:49:25.605Z)

```
WALL_SECONDS constant: 'wall_seconds'
_RATE_MULTIPLIED tuple: ('gpu_seconds', 'wall_seconds', 'egress_gb')
Cost calculation works: 1.0, breakdown: {'wall_seconds': 1.0}
```

20. assistant (2026-06-22T09:49:27.698Z)

Perfect. Let me now check the full integration path one more time to ensure there are no defects:

21. user (2026-06-22T09:49:28.718Z)

```
494     billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495     cost = cost_report(
496         billed_seconds=billed,
497         machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498         fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499         gpu_seconds=total_wall,
```

22. assistant (2026-06-22T09:49:31.018Z)

Now let me verify there are no issues with the constant, including checking if it's exported and accessible from the module:

23. user (2026-06-22T09:49:39.248Z)

```
? All billing integration checks PASSED
- WALL_SECONDS constant: 'wall_seconds'
- Included in _RATE_MULTIPLIED: True
- Cost calculation: 3600.0s @ $0.35/hr = $0.350000 / ?29.05
- Breakdown: {'wall_seconds': 0.35}
```

24. assistant (2026-06-22T09:49:42.940Z)

Perfect. Now let me verify there are no other potential issues by checking if the function signatures and usage all match up correctly:

25. user (2026-06-22T09:49:48.138Z)

```
compute_cost_usd signature: {'usage': typing.Mapping[str, float], 'rates': typing.Mapping[str, float], 'return': typing.Tuple[float, typing.Dict[str, float]]}
cost_report signature: {'billed_seconds': <class 'float'>, 'machine_usd_per_hr': <class 'float'>, 'fx_inr_per_usd': <class 'float'>, 'gpu_seconds': <class 'float'>, 'return': typing.Dict[str, typing.Any]}
to_inr signature: {'cost_usd': <class 'float'>, 'fx_inr_per_usd': <class 'float'>, 'return': <class 'float'>}
```

```
Full cost_report result:
  billed_seconds: 3600.0
  gpu_seconds: 1800.0
  machine_usd_per_hr: 0.35
  usd: 0.35
  inr: 29.05
  breakdown: {'wall_seconds': 0.35}
```

? All function signatures and calls are correct

26. assistant (2026-06-22T09:49:50.972Z)

Excellent. Let me perform one final check to ensure there are no edge cases or production issues:

27. user (2026-06-22T09:50:00.320Z)

```
Exit code 1
? Edge case 1 (zero seconds): PASS
? Edge case 2 (zero rate): PASS
? Edge case 3 (large numbers): PASS - cost: $138.8889
Traceback (most recent call last):
  File "<stdin>", line 24, in <module>
AssertionError: Breakdown precision: expected 0.0001, got 0.0
```

28. assistant (2026-06-22T09:50:02.217Z)

Interesting! This reveals a potential issue. Let me investigate further:

29. user (2026-06-22T09:50:07.413Z)

```
Cost report result: {'billed_seconds': 1.0, 'gpu_seconds': 0.0, 'machine_usd_per_hr': 0.0001, 'usd': 0.0, 'inr': 0.0, 'breakdown': {'wall_seconds': 0.0}}
Rate per second: 2.7777777777777778e-08
Usage: {'wall_seconds': 1.0}
Rates: {'wall_seconds': 2.7777777777777778e-08}
USD: 0.0, Breakdown: {'wall_seconds': 0.0}
Manual calculation: 1.0 * 2.7777777777777778e-08 = 2.7777777777777778e-08, rounded to 6 decimals:
```


0.0

30. assistant (2026-06-22T09:50:09.706Z)

This is actually correct behavior - the rounding to 6 decimals is intentional and appropriate for USD. Let me verify that realistic production values work correctly:

31. user (2026-06-22T09:50:20.815Z)

Testing with REALISTIC production values:

Scenario 1: T4 GPU, 1 hour runtime

Billed: 3600.0s
Machine rate: \$0.35/hr
Cost: \$0.3500 / ?29.05
Breakdown: {'wall_seconds': 0.35}

Scenario 2: 2-3 videos, ~600 seconds pipeline time

Billed: 600.0s
Cost: \$0.058333 / ?4.84
Expected: \$0.058333

Scenario 3: Full VM lifecycle (boot+install+pipeline+cleanup)

Total VM uptime: 1800.0s (~30 min)
GPU processing: 600.0s (~10 min)
Cost: \$0.175000 / ?14.52
Expected: \$0.175000

? All realistic production scenarios work correctly

32. assistant (2026-06-22T09:50:22.923Z)

Perfect! Now let me do a final comprehensive validation of the finding claim:

33. user (2026-06-22T09:50:37.385Z)

=== VERIFICATION OF FINDING ===

1. billing.py line 20 (WALL_SECONDS constant):

Line 19: GPU_SECONDS = "gpu_seconds" # GPU-active seconds (the big one) ? rate is per-second, per gpu_type
Line 20: WALL_SECONDS = "wall_seconds" # total wall time (separate from GPU so idle/sync isn't billed as GPU)

2. nightly_reelcount.py lines 73-90 (cost_report function):

Line 73: def cost_report(billed_seconds: float, machine_usd_per_hr: float,
Line 80: usage = {billing.WALL_SECONDS: float(billed_seconds)}
Line 81: rates = {billing.WALL_SECONDS: rate_per_sec}
Line 82: usd, breakdown = billing.compute_cost_usd(usage, rates)

3. billing.py line 39-45 (compute_cost_usd handles WALL_SECONDS):

Line 39: for res in _RATE_MULTIPLIED:
Line 40: amount = float(usage.get(res, 0.0) or 0.0)
Line 41: rate = float(rates.get(res, 0.0) or 0.0)
Line 45: total += cost

4. Functional verification:

Usage dict: {'wall_seconds': 3600.0}
Rates dict: {'wall_seconds': 9.722222222222222e-05}
compute_cost_usd result: USD=0.35, breakdown={'wall_seconds': 0.35}
billing.to_inr(usd, 83.0): ?29.05

? FINDING VERIFIED: billing.WALL_SECONDS is correctly defined at line 20

? and correctly used at lines 80-82 in cost_report()

? and correctly handled by `compute_cost_usd()` at lines 39-45

CONCLUSION: NO DEFECT ? Billing integration is correct and production-safe.

34. assistant (2026-06-22T09:50:39.682Z)

Perfect. Now let me generate the structured output for this verification:

35. user (2026-06-22T09:50:47.653Z)

Structured output provided successfully

36. assistant (2026-06-22T09:50:54.393Z)

Summary

****FINDING VERIFIED AS CORRECT**** ? No defects found.

The finding's claim that "billing.WALL_SECONDS constant exists and `cost_report()` correctly uses it" is ****confirmed to be true**** by adversarial verification:

****Evidence:****

- ****backend/billing.py:20**** exports `WALL_SECONDS = "wall_seconds"` as a module-level constant
- ****backend/billing.py:25**** includes it in `_RATE_MULTIPLIED`, marking it for cost multiplication
- ****backend/billing.py:39-45**** correctly iterates `_RATE_MULTIPLIED` and computes `usage[res] × rates[res]`
- ****scripts/nightly_reelcount.py:80-82**** correctly constructs usage/rates dicts with the constant key and passes them to `billing.compute_cost_usd()`
- ****Functional testing**** confirms accurate calculations: 1 hour @ \$0.35/hr = \$0.35 USD = ?29.05 INR
- ****All realistic scenarios**** (T4 GPU costing, multi-video runs with boot/install overhead) produce correct results to 6 decimal places USD precision

****No production issues, security defects, or regressions**** identified. The billing integration is correct and safe to deploy.